

[Hive] AI 辅助编写 Hive SQL

练习题

习题一 将 Mysql ddl 转为 Hive ddl

下面是一个 Mysql 中的 ddl，你可以尝试将它转为 Hive 中带有分区字段的 ddl。

```
CREATE TABLE `district` (  
  
  `id` bigint(20) NOT NULL AUTO_INCREMENT,  
  
  `create_time` datetime DEFAULT NULL,  
  
  `deleted` bit(1) DEFAULT NULL,  
  
  `update_time` datetime DEFAULT NULL,  
  
  `version` int(11) DEFAULT NULL,  
  
  `area_code` bigint(20) DEFAULT NULL,  
  
  `city_code` varchar(255) DEFAULT NULL,  
  
  `lat` decimal(19,2) DEFAULT NULL,  
  
  `level` int(11) DEFAULT NULL,  
  
  `lng` decimal(19,2) DEFAULT NULL,  
  
  `merger_name` varchar(255) DEFAULT NULL,  
  
  `name` varchar(255) DEFAULT NULL,  
  
  `parent_code` bigint(20) DEFAULT NULL,  
  
  `pinyin` varchar(255) DEFAULT NULL,
```

```
`short_name` varchar(255) DEFAULT NULL,  
  
`zip_code` bigint(20) DEFAULT NULL,  
  
PRIMARY KEY (`id`)  
  
) ENGINE=MyISAM DEFAULT CHARSET=utf8mb4;
```

习题二 编写 SQL 算指标

有一张 hive 表有如下字段，

hl_code, --总公司 ID

hl_name, --总公司名称

comp_id, --分公司 ID

comp_nm, --分公司名称

year_id, --年份

month_id, --月份

yysr, --实际月度营业收入

ydjhz, --计划月度营业收入

ndjhz, --计划年度营业收入

用 hiveSQL 计算出来一张宽表 字段包括(总公司月度收入，总公司计划月度收入，总公司去年同期月度收入，总公司季度收入，总公司计划季度收入，总公司去年同期季度收入，总公司年度收入，总公司去年同期年度收入)

习题三 解释与优化 SQL

下面有一个超长 SQL 等待你去维护


```

SELECT
    id,
    rk,
    `date`,
    DATE_SUB(`date`, INTERVAL rk day) date1
from
(
    SELECT
        id,
        `date`,
        ROW_NUMBER() over (
            PARTITION by id
            ORDER BY
                `date`
        ) rk
    FROM
        demo3
    ) t1
    ) t2
    ) t3
    ) t4
    ) tt1
    LEFT JOIN demo3_red dr on tt1.id = dr.id
    and tt1.`date` = dr.`date`
    ) tt2
    ) tt3
    ) tt4
) ttt1
join (
    SELECT
        seven_day_rate
    from
        (
            SELECT
                id,
                rk,

```

```

        `date`,
        datel,
        rk2,
        max_login,
        add_money,
        modify_money,
        modify_sev_money,
        sum(
            IF(modify_sev_money is not null, 1, 0)
        ) / sum(
            if(add_money is not null, 1, 0)
        ) seven_day_rate
from
    (
        SELECT
            id,
            rk,
            `date`,
            datel,
            rk2,
            max_login,
            add_money,
            modify_money,
            if(
                max_login - rk2 & gt;= 6,
                modify_money,
                null
            ) modify_sev_money
        from
            (
                SELECT
                    id,
                    rk,
                    `date`,
                    datel,
                    rk2,

```

[illegible]

```

        `date`,
        date1,
        ROW_NUMBER() over (partition by id, date1) rk2
FROM
    (
        SELECT
            id,
            rk,
            `date`,
            DATE_SUB(`date`, INTERVAL rk day) date1
        from
            (
                SELECT
                    id,
                    `date`,
                    ROW_NUMBER() over (
                        PARTITION by id
                        ORDER BY
                            `date`
                    ) rk
                FROM
                    demo3
            ) t1
        ) t2
    ) t3
    ) t4
    ) tt1
    LEFT JOIN demo3_red dr on tt1.id = dr.id
    and tt1.`date` = dr.`date`
    ) tt2
    ) tt3
    ) tt4
    ) tt5
    ) ttt2

```


现在，你可以尝试用 GPT 来优化这段 SQL

参考答案

习题一 将 Mysql ddl 转为 Hive ddl

第 1 轮提示

提示

```
CREATE TABLE `district` (  
  
  `id` bigint(20) NOT NULL AUTO_INCREMENT,  
  
  `create_time` datetime DEFAULT NULL,  
  
  `deleted` bit(1) DEFAULT NULL,  
  
  `update_time` datetime DEFAULT NULL,  
  
  `version` int(11) DEFAULT NULL,  
  
  `area_code` bigint(20) DEFAULT NULL,  
  
  `city_code` varchar(255) DEFAULT NULL,  
  
  `lat` decimal(19,2) DEFAULT NULL,  
  
  `level` int(11) DEFAULT NULL,  
  
  `lng` decimal(19,2) DEFAULT NULL,  
  
  `merger_name` varchar(255) DEFAULT NULL,  
  
  `name` varchar(255) DEFAULT NULL,  
  
  `parent_code` bigint(20) DEFAULT NULL,
```

```
`pinyin` varchar(255) DEFAULT NULL,  
  
`short_name` varchar(255) DEFAULT NULL,  
  
`zip_code` bigint(20) DEFAULT NULL,  
  
PRIMARY KEY (`id`)  
  
) ENGINE=MyISAM DEFAULT CHARSET=utf8mb4;
```

请将上面的 ddl 转为 hive 3.0 里可用的 ddl。所有的字段都尽可能地加上建议性的 comment。

表要使用 orc 存储格式，snappy 压缩，并加上一个名为 dt 的分区字段。

答

下是根据您的要求将 MySQL 表 DDL 转换为适用于 Hive 3.0 的 DDL：

```
CREATE EXTERNAL TABLE district (  
  id BIGINT COMMENT '主键 ID',  
  create_time TIMESTAMP COMMENT '创建时间',  
  deleted BOOLEAN COMMENT '是否删除',  
  update_time TIMESTAMP COMMENT '更新时间',  
  version INT COMMENT '版本号',  
  area_code BIGINT COMMENT '区域代码',  
  city_code STRING COMMENT '城市代码',  
  lat DECIMAL(19,2) COMMENT '纬度',  
  level INT COMMENT '层级',  
  lng DECIMAL(19,2) COMMENT '经度',  
  merger_name STRING COMMENT '合并名称',  
  name STRING COMMENT '名称',  
  parent_code BIGINT COMMENT '父级代码',  
  pinyin STRING COMMENT '拼音',  
  short_name STRING COMMENT '简称',  
  zip_code BIGINT COMMENT '邮政编码'  
)  
COMMENT '地区信息表'  
PARTITIONED BY (dt STRING)
```

```
STORED AS ORC
TBLPROPERTIES ('orc.compress'='SNAPPY');
```

这个 DDL 创建了一个名为 **district** 的外部表，并为所有字段添加了建议性的 `comment`。表使用了 ORC 存储格式、Snappy 压缩，并添加了一个名为 **dt** 的分区字段。请注意，由于 Hive 不支持 BIT 类型，因此我将 **deleted** 字段的数据类型从 **bit(1)** 更改为 **BOOLEAN**。

反馈

如果发现 SQL 不是很合适，你还可以继续迭代提示来对 SQL 进行局部修改。

第 2 轮提示

提示

请为刚才的 HiveSQL 加上一个 `location` 的设置，它的路径是 `/warehouse/gmall/`

答

在 Hive 中创建表时，可以通过 **LOCATION** 子句指定表的 HDFS 存储路径。这里是更新后的 Hive DDL，包含了您指定的 **LOCATION** 设置：

```
CREATE EXTERNAL TABLE district (
  id BIGINT COMMENT '主键 ID',
  create_time TIMESTAMP COMMENT '创建时间',
  deleted BOOLEAN COMMENT '是否删除',
  update_time TIMESTAMP COMMENT '更新时间',
  version INT COMMENT '版本号',
  area_code BIGINT COMMENT '区域代码',
  city_code STRING COMMENT '城市代码',
  lat DECIMAL(19,2) COMMENT '纬度',
  level INT COMMENT '层级',
  lng DECIMAL(19,2) COMMENT '经度',
  merger_name STRING COMMENT '合并名称',
  LOCATION '/warehouse/gmall/';
```

```
name STRING COMMENT '名称',
parent_code BIGINT COMMENT '父级代码',
pinyin STRING COMMENT '拼音',
short_name STRING COMMENT '简称',
zip_code BIGINT COMMENT '邮政编码'
)
COMMENT '地区信息表'
PARTITIONED BY (dt STRING)
STORED AS ORC
TBLPROPERTIES ('orc.compress'='SNAPPY')
LOCATION '/warehouse/gmall/';
```

现在，这个表的数据将存储在指定的 HDFS 路径`/warehouse/gmall/`中。请注意，这是一个外部表，因此 Hive 不会在删除表时自动删除 HDFS 上的数据。这有助于在删除和重新创建表时保留数据。

反馈

已经满足需求。

习题二 编写 SQL 算指标

第 1 轮提示

提示

有一张 hive 表有如下字段，

hl_code, --总公司 ID

hl_name, --总公司名称

comp_id, --分公司 ID

comp_nm, --分公司名称

year_id, --年份

month_id, --月份

yysr, --实际月度营业收入

ydjhz, --计划月度营业收入

ndjhz, --计划年度营业收入

用 hiveSQL 计算出来一张宽表 字段包括(总公司月度收入，总公司计划月度收入，总公司去年同期月度收入，总公司季度收入，总公司计划季度收入，总公司去年同期季度收入，总公司年度收入，总公司去年同期年度收入)

答

以下是一个 Hive SQL 查询，用于计算您要求的各个指标。请根据您的实际表名替换

your_table_name:

```
WITH monthly AS (  
    SELECT hl_code, hl_name, year_id, month_id,  
           SUM(yysr) AS yysr_sum, SUM(ydjhz) AS ydjhz_sum  
    FROM your_table_name  
    GROUP BY hl_code, hl_name, year_id, month_id  
)  
,  
quarterly AS (  
    SELECT hl_code, hl_name, year_id, CEIL(month_id / 3) AS quarter_id,  
           SUM(yysr) AS yysr_sum, SUM(ydjhz) AS ydjhz_sum  
    FROM your_table_name  
    GROUP BY hl_code, hl_name, year_id, CEIL(month_id / 3)  
)  
,  
yearly AS (  
    SELECT hl_code, hl_name, year_id,  
           SUM(yysr) AS yysr_sum, SUM(ndjhz) AS ndjhz_sum  
    FROM your_table_name  
    GROUP BY hl_code, hl_name, year_id  
)
```

```

SELECT m.hl_code, m.hl_name, m.year_id, m.month_id,
       m.yysr_sum AS month_yysr, m.ydjh_sum AS month_ydjh,
       lag(m.yysr_sum) OVER (PARTITION BY m.hl_code ORDER BY m.year_id) AS month_yysr_ly,
       q.yysr_sum AS quarter_yysr, q.ydjh_sum AS quarter_ydjh,
       lag(q.yysr_sum) OVER (PARTITION BY q.hl_code ORDER BY q.year_id) AS quarter_yysr_ly,
       y.yysr_sum AS year_yysr, y.ndjh_sum AS year_ndjh,
       lag(y.yysr_sum) OVER (PARTITION BY y.hl_code ORDER BY y.year_id) AS year_yysr_ly
FROM monthly m
JOIN quarterly q ON m.hl_code = q.hl_code AND m.year_id = q.year_id AND CEIL(m.month_id /
3) = q.quarter_id
JOIN yearly y ON m.hl_code = y.hl_code AND m.year_id = y.year_id;

```

这个查询首先创建了 3 个临时表（**monthly**、**quarterly** 和 **yearly**），分别计算每月、每季度和每年的营业收入和计划收入。然后，通过将这些临时表连接在一起，并使用窗口函数 **lag()** 计算上年同期收入，生成了您所要求的宽表。请注意，这个查询使用了窗口函数，因此需要 Hive 0.11.0 或更高版本。

反馈

我们很快就得到了一个 SQL，这个时候我们应当对 SQL 进行充分的测试。如果你当前没有测试环境，那么其实可以让 GPT 来为你生成测试用的模拟数据。

第 2 轮提示

提示

请为刚才的表写一个 hive ddl。并使用 hive sql 向里面插入足够的测试数据

答

首先，这是一个示例 DDL，用于创建包含所需字段的表。请注意，根据您的需求，您可能需要根据实际情况调整数据类型和 COMMENT。

```

CREATE EXTERNAL TABLE company_financials (
    hl_code STRING COMMENT '总公司 ID',

```

```

hl_name STRING COMMENT '总公司名称',
comp_id STRING COMMENT '分公司 ID',
comp_nm STRING COMMENT '分公司名称',
year_id INT COMMENT '年份',
month_id INT COMMENT '月份',
yysr DOUBLE COMMENT '实际月度营业收入',
ydhz DOUBLE COMMENT '计划月度营业收入',
ndjh DOUBLE COMMENT '计划年度营业收入'
)
STORED AS ORC
TBLPROPERTIES ('orc.compress'='SNAPPY');

```

接下来，我们将使用 Hive SQL 插入一些示例数据。请注意，这些数据是随机生成的，您可能需要根据实际情况进行调整。

```

INSERT INTO company_financials (hl_code, hl_name, comp_id, comp_nm, year_id, month_id,
yysr, ydhz, ndjh) VALUES
('HL001', '总公司 1', 'COMP1', '分公司 1', 2021, 1, 10000, 11000, 120000),
('HL001', '总公司 1', 'COMP1', '分公司 1', 2021, 2, 12000, 13000, 120000),
('HL001', '总公司 1', 'COMP1', '分公司 1', 2021, 3, 15000, 16000, 120000),
('HL001', '总公司 1', 'COMP1', '分公司 1', 2021, 4, 18000, 19000, 120000),
('HL001', '总公司 1', 'COMP1', '分公司 1', 2021, 5, 20000, 21000, 120000),
('HL001', '总公司 1', 'COMP1', '分公司 1', 2021, 6, 24000, 25000, 120000),
('HL001', '总公司 1', 'COMP1', '分公司 1', 2021, 7, 26000, 27000, 120000),
('HL001', '总公司 1', 'COMP1', '分公司 1', 2021, 8, 28000, 29000, 120000),
('HL001', '总公司 1', 'COMP1', '分公司 1', 2021, 9, 31000, 32000, 120000),
('HL001', '总公司 1', 'COMP1', '分公司 1', 2021, 10, 33000, 34000, 120000),
('HL001', '总公司 1', 'COMP1', '分公司 1', 2021, 11, 35000, 36000, 120000),
('HL001', '总公司 1', 'COMP1', '分公司 1', 2021, 12, 38000, 39000, 120000),
('HL001', '总公司 1', 'COMP1', '分公司 1', 2022, 1, 11000, 12000, 130000

```

反馈

这个时候回答已经中断了，最后一个 SQL 缺少了);做结尾。现在，如果觉得数据量不够，测试场景不够全面，那么还可以让它继续生成数据。

第三轮提示

提示

继续生成数据，添加一些额外的总公司和分公司

答

以下是一些包含额外总公司和分公司的示例数据。您可以将其添加到先前插入的数据中以进行测试。

```
INSERT INTO company_financials (hl_code, hl_name, comp_id, comp_nm, year_id, month_id,
yysr, ydjhz, ndjhz) VALUES
('HL001', '总公司 1', 'COMP1', '分公司 1', 2022, 2, 13000, 14000, 130000),
('HL001', '总公司 1', 'COMP1', '分公司 1', 2022, 3, 16000, 17000, 130000),
('HL001', '总公司 1', 'COMP2', '分公司 2', 2021, 1, 9000, 10000, 110000),
('HL001', '总公司 1', 'COMP2', '分公司 2', 2021, 2, 11000, 12000, 110000),
('HL001', '总公司 1', 'COMP2', '分公司 2', 2021, 3, 14000, 15000, 110000),
('HL002', '总公司 2', 'COMP3', '分公司 3', 2021, 1, 8000, 9000, 100000),
('HL002', '总公司 2', 'COMP3', '分公司 3', 2021, 2, 10000, 11000, 100000),
('HL002', '总公司 2', 'COMP3', '分公司 3', 2021, 3, 13000, 14000, 100000),
('HL002', '总公司 2', 'COMP4', '分公司 4', 2021, 1, 7000, 8000, 90000),
('HL002', '总公司 2', 'COMP4', '分公司 4', 2021, 2, 9000, 10000, 90000),
('HL002', '总公司 2', 'COMP4', '分公司 4', 2021, 3, 12000, 13000, 90000);
```

这些示例数据包括两个总公司（总公司 1 和总公司 2）以及四个分公司（分公司 1、分公司 2、分公司 3 和分公司 4）。这些数据可以用于测试查询以确保它们可以正确处理多个总公司和分公司的情况。

反馈

现在，我们可以在本地的虚拟机中来测试之前的计算逻辑是否正确了。

习题三 解释与优化 SQL

第一轮提示

提示

下面是一个 HQL。能帮我说一下它干了什么事吗，还有能简化的地方吗。

```
SELECT
    two_day_rate,
    seven_day_rate
from
    (
        SELECT
            two_day_rate
        from
            (
                SELECT
                    id,
                    rk,
                    `date`,
                    date1,
                    rk2,
                    max_login,
                    add_money,
                    modify_money,
                    sum(
                        if(modify_money is not null, 1, 0)
                    )/ sum(
                        if(add_money is not null, 1, 0)
                    ) two_day_rate
```

```

from
(
    SELECT
        id,
        rk,
        `date`,
        datel,
        rk2,
        max_login,
        add_money,
        if(rk2 = max_login, null, add_money) modify_money
from
(
    SELECT
        ttl.id,
        rk,
        ttl.`date`,
        datel,
        rk2,
        max_login,
        add_money
from
(
    SELECT
        id,
        rk,
        `date`,
        datel,
        rk2,
        max_login
from
(
    SELECT
        id,
        rk,
        `date`,

```

```

        datel,
        rk2,
        max(rk2) over(partition by id, datel) max_login
from
(
    SELECT
        id,
        rk,
        `date`,
        datel,
        ROW_NUMBER() over (partition by id, datel) rk2
FROM
(
    SELECT
        id,
        rk,
        `date`,
        DATE_SUB(`date`, INTERVAL rk day) datel
from
(
    SELECT
        id,
        `date`,
        ROW_NUMBER() over (
            PARTITION by id
            ORDER BY
                `date`
        ) rk
FROM
    demo3
    ) t1
    ) t2
    ) t3
    ) t4
) tt1
LEFT JOIN demo3_red dr on tt1.id = dr.id

```

```

        and tt1.`date` = dr.`date`
    ) tt2
) tt3
) tt4
) ttt1
join (
    SELECT
        seven_day_rate
    from
        (
            SELECT
                id,
                rk,
                `date`,
                datel,
                rk2,
                max_login,
                add_money,
                modify_money,
                modify_sev_money,
                sum(
                    IF(modify_sev_money is not null, 1, 0)
                ) / sum(
                    if(add_money is not null, 1, 0)
                ) seven_day_rate
            from
                (
                    SELECT
                        id,
                        rk,
                        `date`,
                        datel,
                        rk2,
                        max_login,
                        add_money,
                        modify_money,

```

```

        if(
            max_login - rk2 & gt;= 6,
            modify_money,
            null
        ) modify_sev_money
from
    (
        SELECT
            id,
            rk,
            `date`,
            datel,
            rk2,
            max_login,
            add_money,
            if(rk2 = max_login, null, add_money) modify_money
        from
            (
                SELECT
                    ttl.id,
                    rk,
                    ttl.`date`,
                    datel,
                    rk2,
                    max_login,
                    add_money
                from
                    (
                        SELECT
                            id,
                            rk,
                            `date`,
                            datel,
                            rk2,
                            max_login
                        from

```

```

(
SELECT
    id,
    rk,
    `date`,
    date1,
    rk2,
    max(rk2) over(partition by id, date1) max_login
from
(
    SELECT
        id,
        rk,
        `date`,
        date1,
        ROW_NUMBER() over (partition by id, date1) rk2
FROM
    (
        SELECT
            id,
            rk,
            `date`,
            DATE_SUB(`date`, INTERVAL rk day) date1
        from
            (
                SELECT
                    id,
                    `date`,
                    ROW_NUMBER() over (
                        PARTITION by id
                        ORDER BY
                            `date`
                    ) rk
                FROM
                    demo3
            ) t1
    )

```

```

        ) t2
    ) t3
) t4
) tt1
LEFT JOIN demo3_red dr on tt1.id = dr.id
and tt1.`date` = dr.`date`
) tt2
) tt3
) tt4
) tt5
) ttt2

```

答

这个 HQL 查询包含了两个子查询。首先，我将逐步解释它们各自的作用，然后描述它们如何组合在一起。

第一个子查询计算了 `two_day_rate`，即每个 `id` 在不同 `date1` 的 `modify_money`（条件：`rk2` 不等于 `max_login`）与 `add_money` 的非空条目之和的比率。

第二个子查询计算了 `seven_day_rate`，即每个 `id` 在不同 `date1` 的 `modify_sev_money`（条件：`max_login-rk2` 大于等于 6）与 `add_money` 的非空条目之和的比率。

这两个子查询的结构非常相似。它们都包含了以下几个操作：

计算每个 `id` 和 `date` 的行号 `rk`。

使用 `DATE_SUB` 函数创建一个新列 `date1`，该列是 `date` 减去行号 `rk` 的结果。

对每个 `id` 和 `date1`，计算新的行号 `rk2`。

计算每个 `id` 和 `date1` 的 `max_login`，即 `rk2` 的最大值。

在 `demo3` 表与 `demo3_red` 表之间执行左连接。

计算 `modify_money`（第一个子查询）和 `modify_sev_money`（第二个子查询）。

计算 two_day_rate 和 seven_day_rate。

最后，这个 HQL 将这两个子查询的结果连接在一起，返回 two_day_rate 和 seven_day_rate。

关于简化： 这个查询的主要问题是它的冗余性。许多操作在两个子查询中都执行了。你可以尝试将这些操作合并到一个子查询中，然后使用聚合函数 SUM 和 IF 计算 two_day_rate 和 seven_day_rate。这将使查询更简洁、更易于阅读。

反馈

ChatGPT 已经帮我们解释了这个 HIVE SQL 的分析逻辑，这是一个 200 行的 SQL，人要读上半天，但是对于 ChatGPT 来说只需 1 分钟。这种情况下，我一般会要求 GPT 把 SQL 优化一版，然后拿去测一下执行结果和原先的是否一致。如果能保持一致，就用新的可读性更好的优 GPT 优化出来的 SQL。

第二轮提示

提示

请帮我写一个更简单的，谢谢

答

```
SELECT
    id, date1,
    SUM(IF(modify_money IS NOT NULL, 1, 0)) / SUM(IF(add_money IS NOT NULL, 1, 0)) AS
two_day_rate,
    SUM(IF(modify_sev_money IS NOT NULL, 1, 0)) / SUM(IF(add_money IS NOT NULL, 1, 0)) AS
seven_day_rate
FROM (
    SELECT
        ttl.id, ttl.`date`, date1, rk2, max_login, add_money,
        IF(rk2 = max_login, NULL, add_money) AS modify_money,
        IF(max_login - rk2 >= 6, add_money, NULL) AS modify_sev_money
    FROM (
        SELECT
```



```

        id, `date`, date1, rk2,
        MAX(rk2) OVER (PARTITION BY id, date1) AS max_login
FROM (
    SELECT
        id, `date`, date1,
        ROW_NUMBER() OVER (PARTITION BY id, date1) AS rk2
    FROM (
        SELECT
            id, `date`, DATE_SUB(`date`, INTERVAL rk DAY) AS date1
        FROM (
            SELECT
                id, `date`,
                ROW_NUMBER() OVER (PARTITION BY id ORDER BY `date`) AS rk
            FROM demo3
        ) t1
    ) t2
    ) t3
    ) tt1
LEFT JOIN demo3_red dr
ON tt1.id = dr.id AND tt1.`date` = dr.`date`
) t4
GROUP BY id, date1;

```

这个简化的查询首先计算了每个 id 和 date 的行号 rk，接着创建了一个新列 date1，该列是 date 减去行号 rk 的结果。然后，对每个 id 和 date1，计算了新的行号 rk2，以及每个 id 和 date1 的 max_login，即 rk2 的最大值。在 demo3 表与 demo3_red 表之间执行左连接后，根据条件计算 modify_money 和 modify_sev_money。最后，使用聚合函数 SUM 和 IF，对每个 id 和 date1 计算 two_day_rate 和 seven_day_rate。

反馈

原先的 SQL 没有使用 IF 函数，从而导致每次需要条件分支的时候，直接重写 where 逻辑并增加一个子查询。现在优化后的 SQL 只有 33 行，比原版 200+行的 SQL 可读性不要好太多。